COMPUTER ORGANISATION

Complete Course Notes with Coloured Vector Diagrams & Worked Arithmetic

Definitions + Diagrams + Comparison Tables + RTL/Assembly + Worked Numbers (Booth, IEEE-754, EAT) | 5 Units

SYLLABUS COVERED

Unit 1 Basic structure of a computer: stored-program (Von Neumann), functional units, register organization (PC, IR, AC, MAR, MDR), addressing modes, bus structures & memory interface	**Unit 2** Control unit: instruction types & formats, fetch/execute cycle in RTL, hardwired vs microprogrammed control, microinstruction sequencing, CU as a state machine	
Unit 3 Computer arithmetic: sign-magnitude / 1s / 2s complement, add/subtract with overflow detection, Booth multiplication (worked), restoring division, IEEE 754 single/double format with manual conversions + FPU reality	**Unit 4** I/O organization: buses (PCIe/USB/SATA), interfaces, programmed vs interrupt-driven vs DMA transfers, interrupt sequence, I/O processor	
Unit 5 Memory organization: hierarchy & SRAM/DRAM/ROM/flash, cache (direct/set-assoc/fully, replacement, write policies), EAT, TLB & virtual memory; multiprocessor interconnection; pipelining & hazards, superscalar/ILP, RISC vs CISC	**Worked** Every numeric topic has a step-by-step example (Booth, 2s overflow, IEEE-754 pack, EAT-speedup formulas)	**How to use** Read in order; redraw diagrams; hand-compute the arithmetic before looking

Self-study & exam-prep guide following the COA syllabus. All diagrams are vector graphics - crisp and colourful on screen and in print.

TABLE OF CONTENTS

Study tip: hand-compute every worked number (Booth, overflow keys, IEEE-754, EAT/speedup) without looking - that is the actual exam skill.

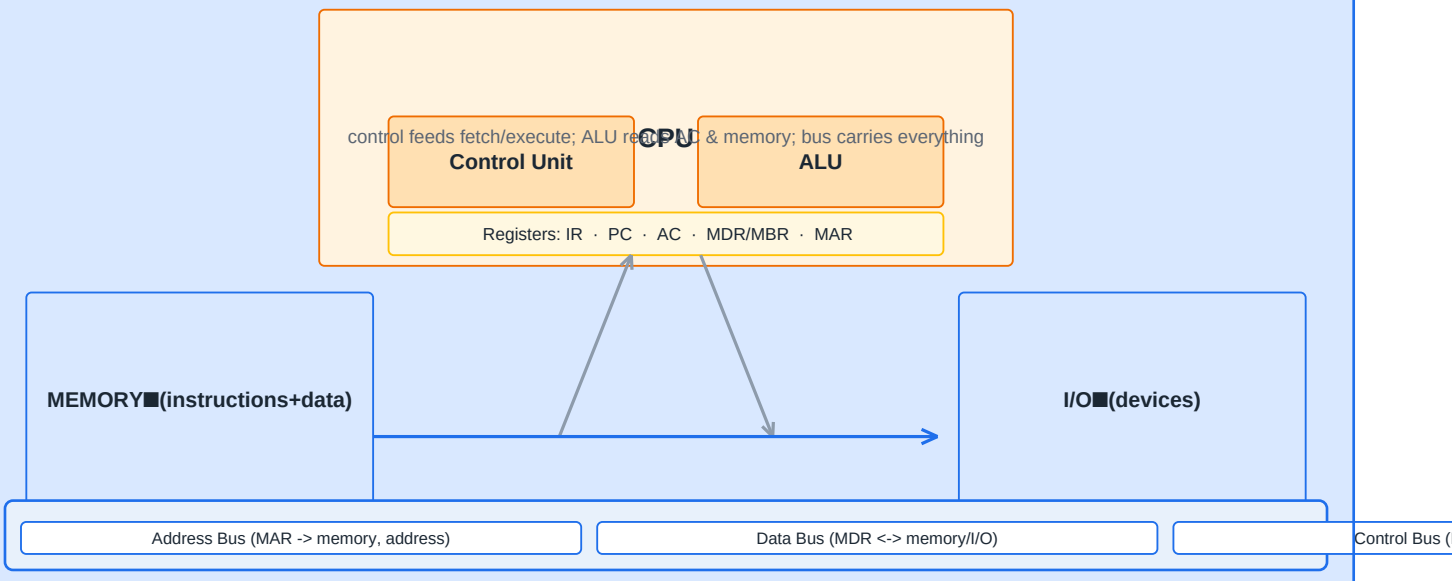
UNIT 1 | BASIC STRUCTURE & FUNCTIONAL UNITS

1.1 Stored-Program Concept & Functional Units

VON NEUMANN STORED-PROGRAM MODEL

Both the PROGRAM (instructions) and its DATA live in the SAME memory; the CPU fetches an instruction, executes it, fetches the next... Sequential by design (control can branch). Five functional units: INPUT, MEMORY, ALU (arithmetic/logic), OUTPUT, CONTROL. This stored-program idea (1945, EDVAC report) turns a machine into a GENERAL-PURPOSE computer.

Von Neumann Computer Structure (Single Bus)



Harvard variant: SEPARATE instruction bus + data bus -> faster, used in DSPs

program counter -> next instruction; MAR/MDR talk to memory; AC is the working accumulator

CPU functional units wired over three buses (address, data, control)

FIVE FUNCTIONAL UNITS AND THEIR DATA PATHS

```
  INPUT (keyboard/scanner)  ->  MEMORY  <-  OUTPUT (display/printer)
       ^                        | |
       |   ALU <-> ACC          | |  reads/writes
       |        |               v v
       +--- CONTROL <--- instruction register -------- PC
  control steps whole show: the FETCH part is hardware + software all wired together
```

THE FETCH-EXECUTE FOUNDATION

CPU registers: PC (program counter - next instruction), IR (instruction register - current instruction), MAR (memory address), MDR/MBR (memory data), AC (accumulator), plus general purpose registers. A program = sequence of instructions; each instruction = {opcode, operand specifiers}. One instruction does: fetch opcode -> decode -> fetch operand(s) -> execute -> store result.

1.2 Register Organization & Addressing Modes

Register	Purpose	Typical width
PC	address of NEXT instruction	log2(membytes)
IR	current fetched instruction	word
MAR	address sent to memory	log2(membytes)
MDR	data to/from memory	word
AC (acc.)	ALU operand + result	word
GPRs R0..Rn	user workhorse registers	word (x86-64: 64 bit)
PSW/Flags	carry, zero, overflow, sign	16-32 bit

Mode	Operand is...	Example	Speed / comment
Immediate	in the instruction	#5	no memory, fast, constant
Register	a register	ADD R1,R2	fastest after immediate
Direct	address in instr	M[20]	one memory access
Indirect	address POINTS to address	(M[20])	slower, 2 accesses
Indexed	reg + offset (base+index)	arr[i]	for arrays/records
Relative	PC + offset	branch +10	position-independent code

addressing.asm

```
; tiny assembly to see addressing modes (MIPS-like pseudo)
lui $t0, 5 # immediate: t0 = 5
add $t1, $t0, $t0 # register: t1 = t0 + t0
lw $t2, 100($t1) # indexed: t2 = M[t1+100]
addi $t1, $t1, 4 # immediate (loop stepping through array)
# high level: arr[ i ] -> base + i*4 (byte address) -> indexed add + lw
```

1.3 Bus Structures & Memory Interface

BUSES

A BUS = shared set of lines: ADDRESS (CPU->mem, sets which location), DATA (both ways, carries the word), CONTROL (RD/WR, timing, interrupts). Single-bus is simple but a bottleneck; separate address/data or hierarchical buses (local, system, expansion, I/O bus like PCIe/USB) help with speed. Bus transaction: MASTER drives address + control; SLAVE/device responds.

Bus	Carries	Typical speed
Local (CPU core)	registers <-> ALU/L1	GHz-scale
System (front side)	CPU <-> memory/controller	multi-GB/s
PCI Express	cards: GPU, NVMe, capture	x16 ~ 16-63 GB/s
USB	peripherals (serial)	USB3 ~ 5-10 Gbit/s
Storage side	SATA / NVMe	6 Gbit/s ~ 3.5 GB/s

REMEMBER

Harvard vs Von Neumann: Harvard has SEPARATE instruction+data buses/memories -> fetch and data access in the SAME cycle (used in DSPs, modern CPU caches are split).

Exercise: *1) Draw the 5 functional units and trace the path for "C = A + B". 2) Convert "add the value in memory location 55 to ACC and store in 56" into 3-address, 2-address, 1-address, 0-address forms. 3) Give one assembly example each of register/direct/indirect/indexed addressing. 4) Why is the bus the "bottleneck" of a single-bus machine?*

UNIT 2 | CONTROL UNIT & INSTRUCTION EXECUTION

2.1 Instruction Types & Formats

INSTRUCTION ANATOMY

An instruction = OPCODE (what to do: ADD, LOAD, BRANCH...) + OPERAND specifiers (where the data is). The NUMBER of operands decides the FORMAT - big trade of code size vs register pressure vs hardware complexity.

Format	Example (semantics)	Size / comment
3-address	ADD R1, R2, R3 (R1 = R2 + R3)	big code, small hardware
2-address	ADD R1, R2 (R1 = R1 + R2)	compact, one operand overwritten
1-address	ADD M[20] (ACC = ACC + M[20])	single accumulator
0-address	stack machines: ADD pops 2, pushes sum	smallest instruction, stack ops
Variable (x86)	opcode + ModRM + imm 1-8 bytes	dense; decode complex (RISC-V fixed 32/16 reduces it)

formats.asm

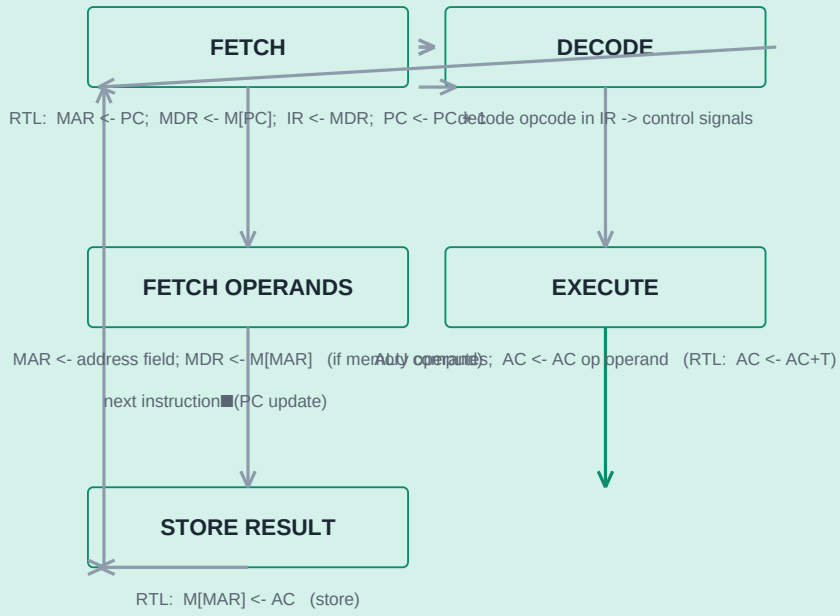
```
; 3-address RISC-V:
add x1, x2, x3 # x1 = x2 + x3
# is that "ADD x1, x2, x3" = R-type: [funct7][rs2][rs1][funct3][rd][opcode]
# 1-address (accumulator) machine doing the same:
LDA a ; ADD b ; STA c # uses hidden ACC -> only one operand in each instr
```

2.2 Control Unit + Instruction Cycle (RTL)

CONTROL UNIT JOB

Translate the fetched instruction into the exact timed micro-actions on every register/data path. Implemented EITHER hardwired (logic gates) OR microprogrammed (control store). The cycle in RTL (Register Transfer Language) is the standard way exams want it written.

Instruction Execution Cycle (fetch -> decode -> execute)



each step is one clock cycle -> total CPI depends on instruction type

fetch-execute cycle with register-transfer (RTL) actions

rtl.txt

```
# canonical 4-step fetch RTL (read memory):
t1: MAR <- PC                   # put address of next instruction
t2: MDR <- Memory,MAR read      # data word arrives  (or M[MAR])
t3: IR <- MDR ; PC <- PC + 1    # instruction in IR, advance PC
t4: decode(IR.OP) -> control    # then operand fetch + execute steps
# execute for  ADD ACC, X  :
t5: MAR <- IR.ADDR ; MDR <- M[X] ;  t6: AC <- AC + MDR ;  t7: M[MAR]<-AC or continue
```

RTL RULE

RTL convention: "M[MAR] <- AC" reads "memory at MAR gets the accumulator". Translation: every register gets <= one new value per clock to avoid races.

2.3 Hardwired vs Microprogrammed Control

Aspect	HARDWIRED control	MICROPROGRAMMED control
Construction	combinational + sequential LOGIC per instruction	ROM/RAM control store of MICROINSTRUCTIONS
Speed	fastest (direct gates)	one micro-step per micro-cycle (slower)
Flexibility	redesign = rewiring	change microcode - no hardware change
Cost / size	cheap in gates once designed	stores + decoder consume space

Aspect	HARDWIRED control	MICROPROGRAMMED control
Errors / patch	hard to fix	microcode patch easy (Intel uses it)
Typical for	RISC: ARM core, RISC-V implementations	CISC: x86 complex instructions

MICROSEQUENCER + CONTROL STORE PRINCIPLE

```
Control store: | microinst1 (control word) | next addr |
sequencing:  microPC -> address-selection -> CONTROL STORE -> control signals
mapping table: machine instr opcode -> start address of its micro-routine
microinst fields: branch flag, SRC/DST select, ALU op, memory rd/wr, next-addr
```

MICROINSTRUCTION DESIGN

Direct vs encoded: encoded packs several control bits into a field decoded later (saves width, adds logic). Horizontal vs vertical: horizontal = wide, one bit per control arrow, executed parallel; vertical = narrow, many microwords with sequencing - like a tiny program.

2.4 Design of a Control Unit

CU AS A FINITE-STATE MACHINE ON THE CLOCK

```
Inputs to CU:   IR (opcode),  flags (Z, N, C, V),  clock
Outputs:        control signals:  PCload,  MEMread,  ALUop,  RegWrite,  MuxSelect...
State machine:  one state per cycle step; next state = f(current, IR.op, flags)
=> every instruction reduces to a FSM over its RTL steps
```

Exercise: *1) Write FETCH + 2 execute instructions (an ADD and a conditional branch) in RTL. 2) Table of 6 differences hardwired vs microprogrammed. 3) Why does x86 need your microcode trick? 4) Stack (0-address) machine: compile "y = a*b+c" to its opcode list.*

UNIT 3 | COMPUTER ARITHMETIC

3.1 Signed Number Representations

Scheme	4-bit examples	Zeros	Adds easily?
Sign-magnitude	+5 = 0101, -5 = 1101	+0 and -0 both	no - needs sign logic
1's complement	+5 = 0101, -5 = 1010	+0 and -0	end-around carry penalty
2's complement	+5 = 0101, -5 = 1011	ONE zero	YES - plain binary add

2'S COMPLEMENT WINS - WHY

Negative = invert all bits + 1 (e.g. 5 = 0101 -> invert 1010 -> +1 = 1011). Addition works with ordinary binary adder: 5 + (-5) = 0101 + 1011 = 10000 -> 4 bits -> 0000 = 0. One representation of zero, range -2^(n-1) .. +2^(n-1)-1 (4-bit: -8..+7).

twoscomp.txt

```
# why the ADD overflow test is exact:
# add two POSITIVE -> result negative means overflow (C=0 but V=1)
# 4 + 7 = 11 > 7 range: 0100 + 0111 = 1011 (looks like -5) -> OVERFLOW flag = 1
# good: -3 + 5 = 2 : 1101 + 0101 = 10010 -> 0010 = 2 (carry out is IGNORED)
# rule: V = sign(Carries out of the top two positions differ)
```

Booth Multiplication Algorithm (-7 x 3 = -21)

M = -7 (11111001 two comp) | Q = 3 (00000011) | A = 0000, Q-1 = 0, count = 4

10 -> A = A - M ; 01 -> A = A + M ; 00/11 -> no op

then ARITHMETIC right shift (A, Q, Q-1)

init	0000	0011	0
A = A - M	0111	0011	0
ashr	0011	1001	1
10 -> A = A - M	1010	1001	1
ashr	1101	0100	1
00 no op, ashr	1110	1010	0
01 -> A = A + M	0101	1010	0
ashr	0010	1101	0

Booth handles 2s-complement negatives WITHOUT special-case on sign bit

pairs (Q0, Q-1): 01 add M, 10 subtract M, 00/11 just shift

result Q,A = 1101,0101 = 2 comp -> -21 (sign handled automatically)

Booth: encoding 0/1 pairs cuts the number of adds for long strings of 1s

BOOTH MEMORY

Booth pairs: look at (Q0, Q1-minus): 01 add M; 10 subtract M; 00 or 11 just arithmetic shift. Count cycles: still shift every cycle, adds only on boundaries.

3.2 Binary Multiplication & Division

MULTIPLY (SHIFT-AND-ADD) & RESTORE DIVISION

MULTIPLY = repeat: test multiplier LSB; if 1 add multiplicand shifted + shift partial product right; hardware = 3 registers (M, Q, A) - each iteration one add + shift, n iterations for n bits. DIVIDE (restoring): shift dividend left bit into remainder, try subtract, if negative restore (add back) and quotient bit 0, else 1; identical register shapes - add/sub + shift turned around.

Operation	Hardware action	Iterations	Result registers
6 x 3 = 18 (unsigned)	A=0; bit3 of Q=1 -> A<=A+M then shift	n	A hold high, Q low
restoring divide 7/2 = 3 r1	rem shift left; rem<=rem-M; if neg restore	shift, rem, Q bits	Q quotient, A remainder

3.3 Floating Point - IEEE 754

THE FORMAT

Value = (-1)^S x 1.M x 2^(E - bias). Single (32-bit): S1 E8 M23, bias 127. Double (64-bit): S1 E11 M52, bias 1023. Exponent all-1s = inf/NaN; all-0 exponent with M=0 = zero (denormal near zero). Computers do FP in hardware FPU.

fp.py

```
# 0.1 + 0.2 in binary FP (why print shows 0.30000000000000004)
0.1 = 1.100.. x 2^-4 (infinite binary tail), rounded to 53 bits
0.2 = 1.100.. x 2^-3
# normalise, add mantissa, re-normalise -> the 0.30000000000000004 appears
>>> 0.1 + 0.2
0.30000000000000004 # Python float = IEEE 754 double
>>> import struct; struct.pack('>f', -6.5).hex() # C0D00000 = -1.1 x 2^2
# practical rule: never compare floats with == ; use eps.
```

Metric	single (32b)	double (64b)
Sign / Exponent / Mantissa	1 / 8 / 23	1 / 11 / 52
Bias	127	1023
Smallest normal	~1.2e-38	~2.2e-308
Largest	~3.4e38	~1.8e308
Precision (decimal digits)	~7	~15-16

PRACTICE CONVERSION

convert -6.5: sign1, |6.5| = 110.1 = 1.101 x 2^2 -> E = 2+127 = 129 = 10000001, M = 101... => 1 10000001 1010000...0 = C0D00000. Do 2-3 of these by hand before the exam.

Exercise: *1) -3 (4-bit 2s) + 6 in binary; overflow test. 2) Booth compute 4 x -3. 3) restoring divide 11/3 by hand. 4) IEEE-754-pack 0.5 and -12.0 by hand; decode 0x3F800000.*

UNIT 4 | I/O ORGANIZATION & DATA TRANSFER

4.1 I/O Bus & Interfaces

WHY I/O IS HARD

Devices differ wildly in speed (mouse ~Hz vs NVMe GB/s), data unit (character vs block), and electrical interface. The OS isolates this: CPU sees a uniform I/O INTERFACE per peripheral - a small set of REGISTERS (status, control, data) and an interrupt line. The interface sits on an I/O BUS (PCIe/USB...) far from the CPU bus, so transfers go: CPU -> bus bridge -> controller -> device.

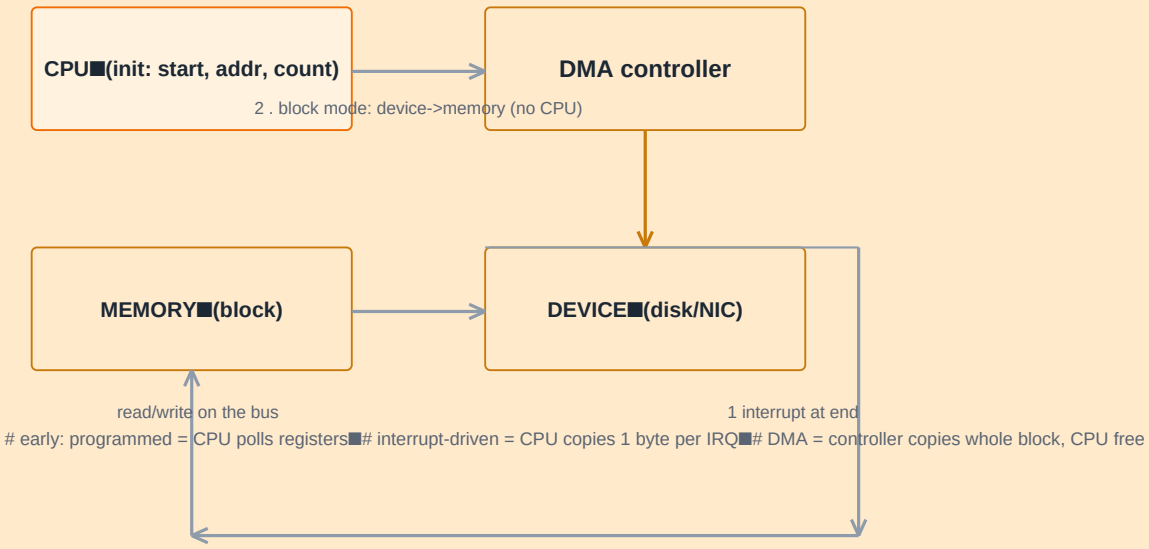
Interface	Data unit	Speed	Type
PCIe (svelte lanes)	packets, x1-x16 lanes	GB/s, hot-plug	internal cards
USB 3.x	packets, serial	5-20 Gbit/s	external near/far
SATA / NVMe	blocks	6 Gbit/s / ~3.5 GB/s	storage
Legacy (PS/2, LPT)	bytes	slow	keyboard/printer

CPU CENTRED I/O BUS TREE

```
CPU --- system bus ---|-->  memory
                      |-->  I/O interface 1 (disk)
                      |-->  I/O interface 2 (NIC)
every interface: [status reg][control reg][data regs]
port-mapped I/O (IN/OUT instr)  vs  memory-mapped I/O (normal loads/stores to address range)
```

4.2 Data Transfer Modes

DMA Transfer (block, one interrupt at end)



DMA steals bus cycles (cycle stealing) - why it only wins for BLOCK transfers

DMA vs programmed vs interrupt I/O - who actually moves the bytes

INTERRUPT HANDLING SEQUENCE

1) device raises INT; 2) CPU finishes current instruction, saves PC+flags; 3) vector table gives ISR address; 4) save more regs, run ISR; 5) restore and execution continues. Priorities and masks make spooled/interrupt storms manageable.

DMA VARIANTS

CYCLE STEALING: DMA grabs the bus for ONE transfer at a time (CPU delayed a cycle). BLOCK: DMA holds bus for the whole block - fastest for bulk but CPU starved. FLY-BY (bus mastering): controller routes data directly device<=>memory without staging in the controller.

4.3 I/O Organisation Design Points

Technique	Mechanism	CPU involvement
Polled (programmed)	CPU loops on status register	100% during transfer
Interrupt-driven	device pokes CPU when ready	1 word per interrupt
DMA single	controller copies one word	set-up + finish
DMA block / channel	controller runs a whole transfer (loop, it can even do linked list)	one interrupt per block
I/O processor (IOP)	separate mini-CPU runs I/O programs	almost none

REVISION LINE

Key idea for MCQs: interrupt-driven good for SLOW devices; DMA essential for HIGH-SPEED BLOCK devices; buffering double to hide speed mismatch.

Exercise: *1) Trace a keyboard press: device -> bus -> interface -> CPU, naming registers and interrupts. 2) Compare programmed vs interrupt vs DMA in a 4-row table. 3) When is cycle stealing safer than block mode? 4) memory-mapped vs port-mapped I/O - two advantages each.*

UNIT 5 | MEMORY, PIPELINING & RISC/CISC

5.1 Memory Hierarchy & Semiconductor Memory

WHY HIERARCHY

One memory cannot be big, fast and cheap. Hierarchy trades: L1 cache (fastest, ~ns, SRAM) -> L2/L3 -> RAM (DRAM, ~50-100 ns) -> SSD -> disk. Locality (temporal - reuse data soon; spatial - use neighbouring data) is what makes caching work: hit ~1 cycle, miss = next level~100 cycles.

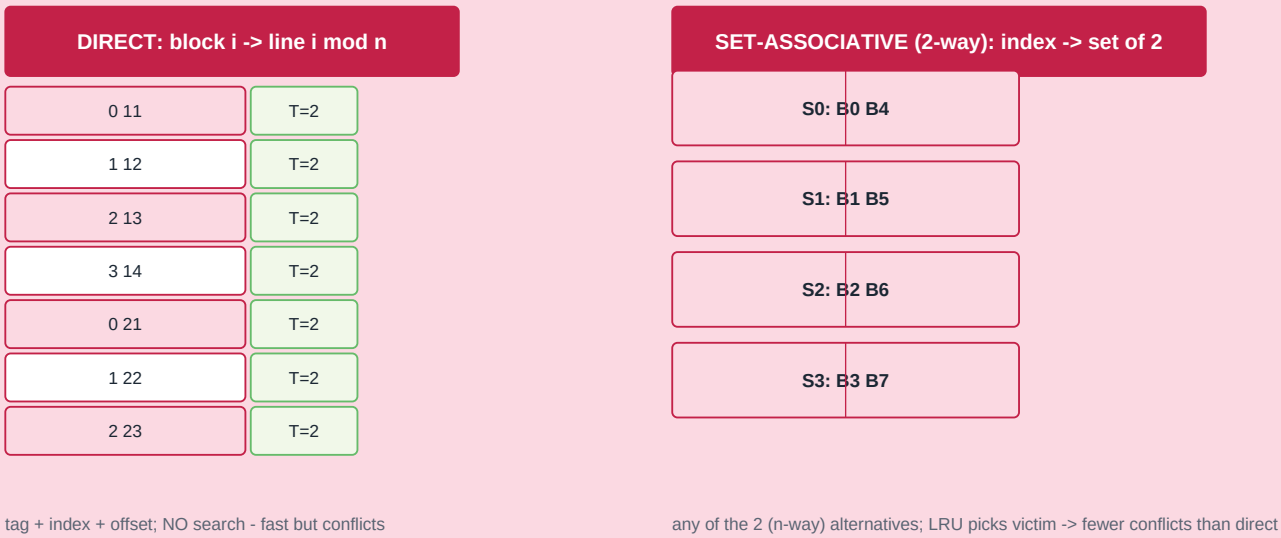
MEMORY PYRAMID WITH LATENCY + SIZE PER LEVEL

```
registers (ns, KB)  <-- L1 (1-4 cyc, tens of KB)  <-- L2 (ns->tens)  <-- L3 (shared)
     <-- DRAM main memory (GB, ~50-100 ns)  <-- SSD (multimillion ns!!)  <-- disk
every hop up = ~10x capacity and ~10x latency; hit rates 90%+ at L1 for typical loops
```

Type	Technology	Volatile?	Use
SRAM (cache)	6-transistor cells	yes	L1/L2/L3 - fast, little density
DRAM (RAM)	1T+cap, refresh	yes (needs refresh)	main memory GBs
ROM	mask/EPROM/EEPROM (BIOS/UEFI)	no	firmware, boot
Flash (SSD)	floating gate	no	storage, boot, USB

5.2 Cache Organization

Cache Mapping: Direct vs Set-Associative



tag + index + offset; NO search - fast but conflicts

any of the 2 (n-way) alternatives; LRU picks victim -> fewer conflicts than direct

write-back vs write-through: through = update memory always (slow, safe); back = update cache, write later (fast, dirty bit)

replacement: LRU, FIFO, random; hit on access when tag matches + valid bit

direct needs no search but blocks staunch; sets keep search small

AT A GLANCE - NUMBERS

Direct: block# mod #lines -> each memory block has ONE possible line -> conflicts. Fully associative: any block anywhere, search all tags - no conflicts, slow. SET-associative (2/4/8-way): index chooses a SET of n lines -> only n tags searched vs 1 for direct. Replacement when set full: LRU, FIFO, random. Write: write-through (memory always updated, simple, slow) vs write-back (update only cache, dirty bit, mark for flush).

eat.txt

```
# memory-pattern roles: effective access time is the markov sum
EAT = H * Tc + (1-H) * (Tc + Tm)          # H = hit ratio
# 90% hits, cache 5 ns, main 60 ns  -> EAT = 0.9*5 + 0.1*65 = 11 ns (< 12 ns budget?)
```

5.3 Virtual Memory & TLB

VIRTUAL MEMORY RECAP

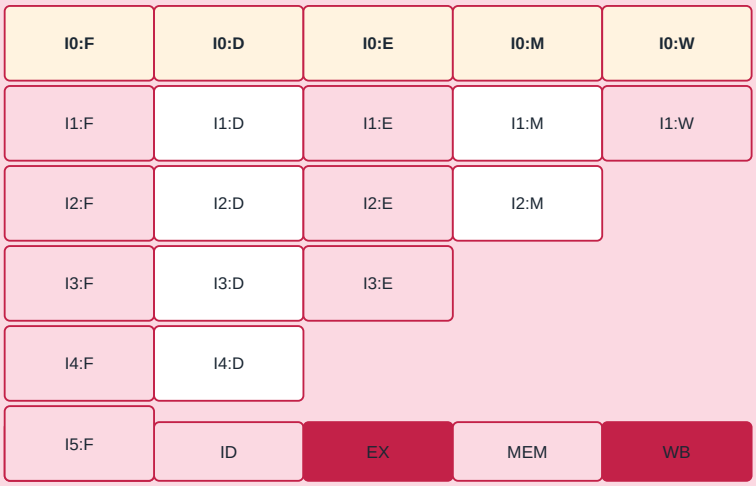
Each process gets a full virtual address space; hardware TLB caches virtual->physical page mappings, PTE bits V/R/W; on miss -> page table walk; on invalid -> page fault into the OS. Two-level tables, huge pages, ASID tags. Cache + TLB interplay: address goes TLB first, then cache; a "curse" either order works fine in practice with both.

TLB + PAGE-TABLE WALK -> VIRTUAL MEMORY MACHINERY

```
virtual addr [VPN | in-page offset]  --TLB-->  [PFN | offset] = physical
translation miss -> walk page table (4 levels x86-64) -> fill TLB
invalid PTE -> page fault -> OS bring page from disk -> reload TLB + retry
```

5.4 Pipelining, ILP, RISC vs CISC

5-stage Instruction Pipeline (ideal, no hazards)



steady state: 1 instruction finishes per cycle -> speedup ~ #stages (ideal)

speedup = CPI_combo/CPI_pipe; CPI_pipe = 1 + stall_cycles + flush_cycles (pizza of hazards)

pipeline: F-D-E-M-W with overlap, then hazards

HAZARDS

STRUCTURAL: two instrs want same unit (fix: separate ports / stall). DATA: RAW (dependent - forwarding from EX/MEM stage reduces stalls; load-use stalls 1). CONTROL: branch - predict (2-bit history) or flush (penalty = pipeline length at each mispredict). CPI_pipe = 1 + stall + flush. Superscalar = issue 2-4 instr per cycle (ILP); OOO = run ahead; SIMD = one op many lanes (like x86 SSE/AVX).

RISC WINS AND CISC SURVIVES

RISC philosophy: fixed formats, load/store, hardwired(pipelined), big register file - pipelining is easy; CISC survives via micro-op translation (Intel builds RISC-like ROPs internally). ARM conquered mobile; RISC-V is free/open; x86 keeps the desktop/datacenter software base.

FORMULAS

Pipeline speedup = depth/(1+stall+flush). Amdahl's law: speedup bounded by the serial part: S = 1 / ((1-f)+f/p). Both are fine the exam.

Exercise: *1) Ranking of latency: L1 vs DRAM vs SSD. 2) Direct vs 2-way set vs fully assoc for block 10, 4 lines - where does it live? 3) EAT with H=0.85 Tc=3ns Tm=50ns. 4) Classify hazards on "lw $t0,0($t1); add $t2,$t0,$t3". 5) Amdahl: parallel 80% of a 10x job.*